

Analysis: Fairies and Witches

Fairies and Witches: Analysis

The problem asks us to choose a subset of edges of a graph such that no two edges share the same vertex and these edges can be used to form a convex polygon with non-zero area.

There are two key observations:

1. The number of ways to choose a subset of edges such that no two edges share same vertex is fairly small. In fact, for a complete graph with 15 nodes, the total number of such pairings is 10349536.
2. A convex polygon with non-zero area can be formed with sides of length $l_1, l_2, \dots, l_k$ if and only if $k \geq 3$ and $2 * \max(l_1, l_2, \dots, l_k) < \text{sum}(l_1, l_2, \dots, l_k)$.

Let's try to prove the upper bound on the total number of different pairings. Let's consider the case in which the total number of nodes (N) is even and we have exactly $N/2$ pairings; let us denote the total number of ways to create these $N / 2$ pairings as $f(N)$.

If we try to generate all possibilities via brute-force code, then one strategy can be to choose the minimum numbered (starting from 1) unused vertex and try to pair it with all the remaining vertices (see code in later section). We can say that at every level when we are forming a new pairing we are choosing one vertex of the pairing in exactly one way (i.e. minimum numbered (starting from 1) unused vertex) and the other vertex of pairing can be any of the remaining vertices. For example, if the unused vertices are (2,5,6,8,9,10), then we start by choosing 2 (the minimum number) as one vertex, and then we try to pair it with all 5 of the remaining vertices. Whenever we choose a particular pair, we mark both vertices as used, and then we recursively pair up the rest. In mathematical terms, we can represent the total number of ways to form such pairings as follows:

$$f(N) = (N - 1) * (N - 3) \dots * 1$$

But in our problem there can be any number of edges in final polygon, not just $N/2$. It might be a good idea to try to derive a more general expression before you keep reading!

So we can calculate the total pairings by choosing any set of an even number of vertices and pairing them all up, which gives us the total number as:

$g(N) = \sum C(N, i) * f(i)$, where i is even and $C(N, i)$ is the number of ways of choosing i objects out of N different objects.

Small dataset

We can observe that for $N = 6$, we must use all 6 vertices so that we can have 3 edges, since a valid polygon must have at least 3 edges. All possibilities can be generated by enumerating all $6!$ permutations of arranging 6 numbers, and using the pairs of the first and second, third and fourth, and fifth and sixth numbers to get edges between those node numbers (if they exist).

Our C++ function for checking whether a polygon is valid can be something like this:

```
bool isGoodPolygon(const vector &edges) {  
    if(edges.size() < 3) {
```

```

        return false;
    }
    int tot = 0;
    int mx = 0;
    for(int i = 0; i < edges.size(); i++) {
        tot += edges[i];
        mx = max(mx, edges[i]);
    }
    return tot - mx > mx;
}

```

The overall time complexity is $O(N!)$.

Large dataset

The Large dataset uses the same overall strategy, but we need an efficient way to code the intended brute-force solution. One way of doing that is the following C++ code, which is using [bit masks](#) to store whether a vertex is used or not.

```

int n;
int L[N][N];
// Whether the i-th bit of mask is on.
int bit(int mask, int i) {
    return (mask >> i) & 1;
}

int solve(int mask, vector &edges) {
    int res = 0;
    if(mask + 1 == (1 << n)) {
        res = isGoodPolygon(edges);
        return res;
    }

    for(int i = 0; i < n; i++) if(!bit(mask, i)) {
        res += solve(mask | (1 << i), edges);
        for(int j = i + 1; j < n; j++) if(!bit(mask, j) && L[i][j]) {
            edges.push_back(L[i][j]);
            res += solve(mask | (1 << i) | (1 << j), edges);
            edges.pop_back();
        }
        break;
    }
    return res;
}

```

The overall time complexity is $O(g(N) * N)$.

Analysis: Kickstart Alarm

Kickstart Alarm: Analysis

The problem asks us to calculate the summation of power of each wakeup call:

$POWER_1 + POWER_2 + \dots + POWER_K$, where $POWER_i$ is just the summation of the i -th exponential-power of all the contiguous subarrays of the Parameter Array.

Small dataset

For Small dataset, we can iterate over every subarray of the given array and calculate the summation of $POWER_i$ for all $i \leq K$. Thus, the simplest brute solution will work for Small dataset.

Pseudocode for Small dataset:

```
result = 0
for(k in 1 to K) {
  for(L in 1 to N) {
    for(R in L to N) {
      for(j in L to R) {
        result = result + A[j] * pow(j-L+1,k)
        result %= 1000000007
      }
    }
  }
}
```

In the above pseudocode, we can precompute all the $pow(a, b)$ values for $1 \leq a \leq n$ and $1 \leq b \leq k$.

The overall time complexity is $O(N^3 \times K)$.

Large dataset

The above solution will not work for Large dataset. To solve for Large dataset, let's iterate over every position x and calculate the contribution by A_x to the result for all subarrays where this element is y -th element in the subarray.

- If $y > x$, there is no subarray such that A_x can be y -th element.
- For $y \leq x$, there is exactly one index where the subarray must start (i.e $y - 1$ places before x). Hence, all the subarrays starting at $(n - (y - 1))$ and ending on or after index x will have A_x at position y in the subarray. Therefore, the number of subarrays with element A_x at y -th position in the subarray will be $(n - x + 1)$.

Contribution from this element as y -th element in one subarray =

$$A_x \times y^1 + A_x \times y^2 + \dots + A_x \times y^K.$$

Let us denote with $S(x, y)$ as the contribution from this element as y -th element in all subarrays. Combining above observations, we can show that

$$S(x, y) = (n - x + 1) \times A_x \times (y^1 + y^2 + \dots + y^K).$$

- $S(x, y) = 0$ for $y > x$.
- $S(x, y) = A_x \times K \times (n - x + 1)$ for $y = 1$.

- $S(x, y) = \frac{(n-x+1) \times A_x \times y \times (y^K - 1)}{(y-1)}$ for $y \leq x$ and $y > 1$.

Contribution by element at position x to the result (let us say $C(x) = \sum S(x, y)$ for $1 \leq y \leq n$)
 $= (n - x + 1) \times A_x \times \left(K + \frac{2 \times (2^K - 1)}{(2-1)} + \frac{3 \times (3^K - 1)}{(3-1)} + \dots + \frac{x \times (x^K - 1)}{(x-1)} \right)$.

So we can find the contribution by element at position x in $O(N \times \log(K))$. This gives us a $O(N^2 \times \log(K))$ solution to compute contribution of all the elements.

Let us define $G(x) = \frac{C(x)}{(A_x \times (n-x+1))} = K + \frac{2 \times (2^K - 1)}{(2-1)} + \frac{3 \times (3^K - 1)}{(3-1)} + \dots + \frac{x \times (x^K - 1)}{(x-1)}$.

Now if we look closely at $G(x)$ and $G(x+1)$, we can observe that

$$G(x+1) = G(x) + \frac{(x+1) \times ((x+1)^K - 1)}{x}.$$

Hence we can compute $G(x+1)$ from $G(x)$ in $O(\log(K))$ time. And subsequently $C(x+1)$.

Therefore the total time complexity = $O(N \times \log(K))$.

Pseudocode for Large dataset:

```
G[1] = K
C[1] = A[1] * K * n
result = C[1]
for(i in 2 to n){
    // Using the formula derived above to get G[i] from C[i-1]
    G[i+1] = G[i] + i * (i^K - 1) / (i - 1)
    C[i] = G[i] * A[i] * (n - i + 1)

    result = result + C[i]
    result %= 1000000007
}
```

Analysis: Planet Distance

Planet Distance: Analysis

The problem statement explains that we have an undirected connected graph with N nodes and N edges, and exactly one cycle in it. Our task here is to first find the nodes that are part of the cycle, and then find the minimum distance from each of the other nodes to this cycle. From the input, we can form an adjacency list and use this to solve the problem.

Small dataset

We can perform a DFS from every node, keeping track of visited and parent nodes, to get the nodes that are a part of the cycle. During the DFS for node i , if the first visited node we encounter (that is not the parent), is the node i , then node i is part of the cycle.

Time complexity for this method is $O(N^2)$.

Once we have identified all the nodes in the cycle, we can do a BFS from each of these cyclic nodes, keeping track of the number of edges we have travelled so far, to get the minimum distance to all the other nodes. We initially mark all the cyclic nodes as visited, to ensure that we don't traverse the cycle during the BFS.

The overall time complexity is $O(N^2)$.

Large dataset

We can identify which nodes are part of the cycle in many ways. One way is to do a DFS, while keeping track of the parent node. If you encounter a node that is visited, and is not the parent of the previous node, it means that this node is part of the cycle. You can then backtrack using the parent nodes to get all the nodes of the cycle. The time complexity is $O(N)$.

The second way is to recursively remove all vertices of degree 1. This can be done efficiently by storing a map of vertices to their degrees.

Initially, traverse the map and store all the vertices with degree = 1 in a queue. Traverse the queue as long as it is not empty. For each node in the queue, mark it as visited, and iterate through all the nodes that are connected to it (using the adjacency list), and decrement the degree of each of those nodes by one in the map. Add all nodes whose degree becomes equal to one to the queue. At the end of this algorithm, all the nodes that are unvisited are part of the cycle.

The time complexity for this method is $O(N)$.

Once we have identified all the nodes in the cycle, we can do a BFS from each of these cyclic nodes, keeping track of the number of edges we have travelled so far, to get the minimum distance to all the other nodes. We initially mark all the cyclic nodes as visited, to ensure that we don't traverse the cycle during the BFS.

The overall time complexity is $O(N)$.